Technical Documentation:
Interactive Quadratic Equation Solver

Author: Daksh Agarwal

Version: 1.0 | Date: January 23, 2026

Abstract

This document provides the technical specification for a C program that solves quadratic equations of the form $ax^2 + bx + c = 0$. The program features interactive input method selection (console or file) and robust input validation that reports all invalid inputs before proceeding with calculations.

Contents

1 System Overview

1.1 Purpose

The program is a command-line utility designed to calculate the roots of a quadratic equation. Its key features are user interactivity, strong input validation, and comprehensive error reporting, making it a robust tool for its defined purpose.

1.2 Architecture

The application is a monolithic C program with a sequential execution flow. The architecture is defined by a multi-stage process within the `main` function:

1. **Mode Selection:** The program first prompts the user to choose an input method.
2. **Data Ingestion:** Based on the user's choice, coefficients are read as raw strings, either from standard input or a file named `coeffs.txt`.
3. **Validation Stage:** Each input string is passed to a dedicated validation function, `is_numeric()`, to check for validity. The program is designed to check all three coefficients and report every error found.
4. **Conditional Calculation:** The roots are calculated only if all three inputs pass the validation stage. The calculation uses the quadratic formula and handles all three cases based on the discriminant's value (positive, zero, negative).

2 Component Specifications

2.1 Main Function: `main()`

The `main` function serves as the primary controller for the program flow. It manages variable declarations, user interaction, file handling, and orchestrates the calls to validation and calculation logic. A key design element is the use of an `all_inputs_valid` flag, which acts as a gatekeeper to prevent the program from attempting to perform mathematical operations on non-numeric data.

2.2 Validation Function: `is_numeric()`

2.2.1 Signature

```
int is_numeric(const char *str);
```

2.2.2 Description

This function is a critical component for ensuring program robustness. It takes a constant character pointer `str` as input and determines if it represents a valid floating-point number.

- **Return Value:** Returns `1` (true) if the string is numeric, `0` (false) otherwise.
- **Validation Logic:**
  1. Returns false for `NULL` or empty strings.
  2. Allows for an optional single leading sign (+ or −).
  3. Returns false for strings containing only a sign.
  4. Iterates through the string, ensuring all other characters are either digits or a single decimal point.
  5. Returns false if more than one decimal point is found.

3 Algorithm: The Quadratic Formula

The program solves the equation using the standard quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The core of the calculation is the **discriminant**, $\Delta = b^2 - 4ac$, which determines the nature of the roots.

- If $\Delta > 0$: There are two distinct real roots.
- If $\Delta = 0$: There is one real root (or two equal real roots).
- If $\Delta < 0$: There are two complex conjugate roots. The program calculates and displays both the real and imaginary parts of these roots.

A check is also performed to ensure that coefficient $a \neq 0$, as this is a definitional requirement for a quadratic equation.

4 Performance Analysis

4.1 Time Complexity: O(1)

The program's runtime is constant. The number of operations (reading input, a fixed number of string checks, and a few mathematical calculations) does not depend on the magnitude or size of the input coefficients. Therefore, the time complexity is $\mathbf{O(1)}$.

4.2 Space Complexity: O(1)

The memory usage is constant. The program uses fixed-size character arrays (`char[100]`) and a small number of double variables. The memory footprint does not scale with the input values. Therefore, the space complexity is $\mathbf{O(1)}$.

5 Build and Execution

5.1 Dependencies

- A standard C compiler (e.g., GCC, Clang).
- The standard C math library (`math.h`).

5.2 Compilation

The program must be compiled with a linker flag to include the math library.

```
gcc program_name.c -o solver_app -lm
```

The `-lm` flag is essential for linking the `sqrt()` function.